

INFORME TÉCNICO

ODOM

Dashboard de Odometría y Visualización de Trayectorias en Tiempo Real

Materia: Robótica / Sistemas Embebidos

Institución: Universidad de Morelos

Integrante: Edmila Tchisseke Coelho da Cruz Camisa

Fecha: 6 de mayo de 2026

Documento generado a partir del código fuente del proyecto.

1. Introducción

El presente informe documenta el desarrollo del proyecto Wheels Odometry, una aplicación web interactiva diseñada para la visualización en tiempo real de la odometría de un robot móvil de dos ruedas. La odometría es la técnica utilizada en robótica para estimar la posición y orientación de un robot basándose en el conteo de pulsos de encoders instalados en sus ruedas.

El sistema simula el comportamiento de un robot diferencial que alterna entre movimiento rectilíneo y rotación en sentido antihorario (CCW), procesando los datos de los encoders para calcular trayectorias y métricas de desplazamiento que se presentan en un dashboard con tema oscuro estilo terminal de monitoreo industrial.

2. Objetivos

2.1 Objetivo General

Desarrollar una aplicación web full-stack que permita visualizar, monitorear y analizar datos de odometría de un robot diferencial de dos ruedas en tiempo real, aplicando principios de robótica móvil y tecnologías modernas de desarrollo web.

2.2 Objetivos Específicos

- Implementar un servidor backend con Express y TypeScript que exponga una API REST para la lectura de pulsos de encoder.
- Crear un simulador de robot diferencial con ciclos de rotación CCW y avance rectilíneo.
- Desarrollar un dashboard interactivo con React y tema oscuro para la visualización de métricas en tiempo real.
- Calcular posición (x, y), orientación (θ) y distancias recorridas a partir de los ticks de encoder.
- Integrar Chart.js para la representación gráfica de trayectorias y ticks en tiempo real.
- Implementar un sistema de logs de odometría con registro de eventos del sistema.

3. Tecnologías Utilizadas

Tecnología	Versión	Rol en el Proyecto
Node.js + TypeScript	ES2022 / TS 5.8	Entorno de ejecución y tipado estático
Express.js	4.21.2	Servidor backend y API REST
React	19.0.1	Biblioteca de interfaz de usuario
Vite	6.2.3	Bundler y servidor de desarrollo con HMR
Tailwind CSS	4.1.14	Estilos utilitarios y diseño responsivo
Chart.js / react-chartjs-2	4.5.1 / 5.3.1	Gráficas de trayectoria y ticks en tiempo real
shadcn/ui + base-ui	4.6.0	Componentes de interfaz reutilizables
Lucide React	0.546.0	Iconografía del dashboard

Tecnología	Versión	Rol en el Proyecto
Motion	12.23.24	Animaciones de interfaz
Google Gemini AI	1.29.0	Integración de IA generativa (asistente)

4. Arquitectura del Sistema

El proyecto sigue una arquitectura cliente-servidor de página única (SPA), donde el frontend en React consume datos del backend Express mediante peticiones HTTP periódicas a la API REST. Toda la aplicación está escrita en TypeScript, lo que garantiza tipado estático tanto en el servidor como en el cliente.

4.1 Backend — Servidor Express (server.ts)

El archivo server.ts configura un servidor Express en el puerto 3000 que expone el endpoint GET /api/ticks. Este endpoint devuelve en formato JSON el estado actual de ambos encoders: ticks acumulados, distancia recorrida en milímetros y revoluciones.

La conversión de ticks a distancia utiliza la constante:

```
MM_PER_TICK = (π × 140) / 1450
```

Que corresponde a una rueda de 140 mm de diámetro y 1450 ticks por revolución completa.

4.2 Simulador de Robot Diferencial

El servidor implementa un simulador de movimiento con intervalo de 50 ms que alterna entre dos estados. En la versión actual del servidor, la rotación está configurada para girar en sentido antihorario (CCW): la rueda izquierda retrocede (−0.5 ticks) y la derecha avanza (+0.5 ticks). El avance rectilíneo incrementa ambas ruedas en +1.5 ticks por ciclo.

Estado	Duración	Ticks L / R	Efecto
ROTATING	~3 s (60 ciclos)	−0.5 / +0.5	Giro CCW sobre el eje del robot
MOVING	~4 s (80 ciclos)	+1.5 / +1.5	Avance rectilíneo hacia adelante

4.3 Frontend — Dashboard React

El cliente React realiza polling al endpoint /api/ticks en intervalos regulares, aplica las ecuaciones de odometría diferencial para calcular la posición (x, y) y orientación (θ) del robot, y actualiza en tiempo real los paneles de métricas, la gráfica de trayectoria cartesiana, la gráfica de ticks y el panel de logs del sistema.

5. Descripción del Dashboard

La interfaz presenta un tema oscuro de alta densidad de información inspirado en sistemas de monitoreo industrial y terminales de robótica. Se organiza en los siguientes paneles:

5.1 Panel de Sensores / Velocidad

Muestra las velocidades calculadas del robot: velocidad lineal (V) en mm/s, velocidad angular (Ω) en rad/s, y el período de muestreo DT en ms. Estos valores se derivan de la diferencia entre lecturas consecutivas de los encoders.

5.2 Paneles MOTOR_A y MOTOR_B

Cada rueda tiene su propio panel de monitoreo que muestra los ticks acumulados, la distancia total recorrida en milímetros y el número de revoluciones completas calculado como ticks / 1450.

5.3 Panel de Localización / Pose

Presenta la posición estimada del robot en el plano cartesiano: coordenada X (mm), coordenada Y (mm) y el heading θ (rad). Estos valores son el resultado de integrar las ecuaciones de odometría diferencial con cada lectura del encoder.

5.4 Mapa de Trayectoria Cartesiana

Gráfica de dispersión (scatter plot) con Chart.js que dibuja en tiempo real el camino recorrido por el robot sobre el plano XY en milímetros. Un marcador triangular amarillo indica la posición actual y orientación estimada del robot. Los ejes están graduados en mm y el plano incluye una cuadrícula de referencia.

5.5 Gráfica de Ticks en Tiempo Real

Gráfica de líneas que muestra la evolución temporal de los ticks acumulados de ambas ruedas: la rueda izquierda en azul (ticks L) y la derecha en naranja (ticks R). Las líneas punteadas indican los valores actuales y permiten visualizar divergencias entre las ruedas durante las fases de rotación.

5.6 Panel de Odometry Logs

Consola de eventos del sistema que registra con marca de tiempo las acciones del robot: `BOOT_SEQUENCE_COMPLETE`, `PID_CONTROLLER_ENGAGED`, `STREAMING_DATA: OK` y actualizaciones de posición `POS_UPDATE` con las coordenadas actuales. Incluye botones `RESET POSE` y `CLEAR PATH` para controlar el estado de la simulación.

6. Resultados

La siguiente captura muestra el dashboard de Wheels Odometry en funcionamiento, con el robot en una posición estimada de X : 425.53 mm, Y : 1003.39 mm y un heading de 2.3437 rad. La rueda izquierda acumula 3,442 ticks (1044.20 mm) y la derecha 5,821 ticks (1765.81 mm), diferencia que refleja las fases de rotación CCW ejecutadas por el simulador.

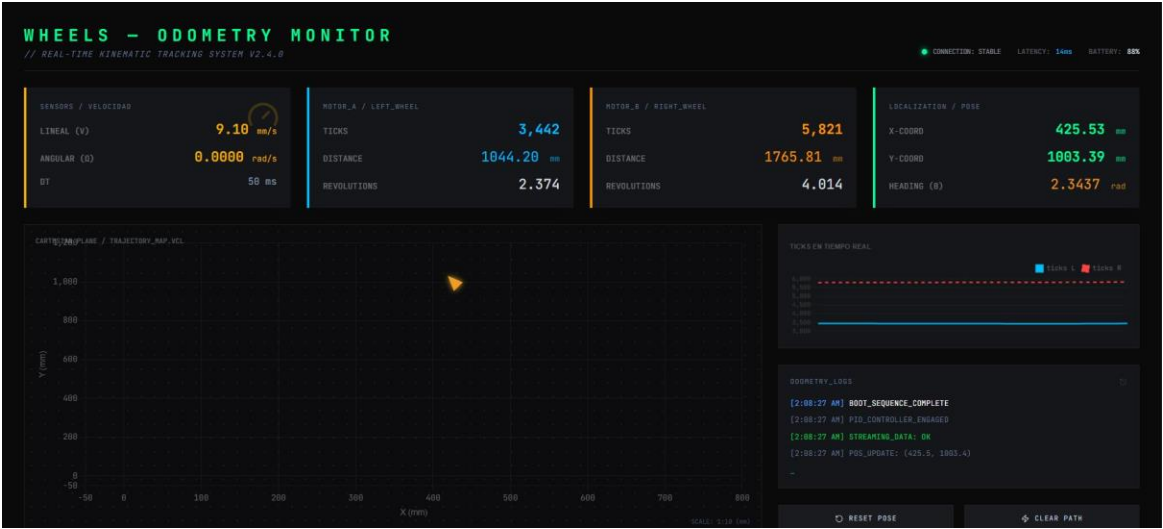


Figura 1. Dashboard Wheels Odometry — Monitor de odometría en tiempo real con tema oscuro estilo terminal.

El mapa cartesiano muestra la trayectoria generada por el patrón de movimiento del simulador (alternar rotación CCW y avance rectilíneo). La gráfica de ticks en tiempo real evidencia la divergencia entre ambas ruedas durante las fases de giro, y la convergencia paralela durante el avance recto. El indicador de conexión en la esquina superior derecha confirma comunicación estable con latencia de 14 ms.

7. Estructura del Proyecto

Archivo / Carpeta	Descripción
server.ts	Servidor Express, API REST y simulador de robot diferencial
src/	Código fuente del frontend React + TypeScript
src/main.tsx	Punto de entrada de la aplicación React
vite.config.ts	Configuración de Vite, plugins y variables de entorno
tsconfig.json	Configuración del compilador TypeScript (target ES2022)
package.json	Dependencias y scripts del proyecto
components.json	Configuración de shadcn/ui — tema base-nova
index.html	Plantilla HTML raíz (título: ODOM)
.env.example	Plantilla de variables de entorno (GEMINI_API_KEY)

8. Fundamentos de Odometría

La odometría por encoder de ruedas es un método de navegación con punto muerto (dead reckoning) que integra las velocidades de las ruedas para estimar el desplazamiento acumulado del robot diferencial.

8.1 Distancia recorrida por rueda

```
d_left  = ticks_left  × MM_PER_TICK  
d_right = ticks_right × MM_PER_TICK
```

8.2 Desplazamiento y giro

```
d_center = (d_left + d_right) / 2      ← avance lineal  
delta_θ  = (d_right - d_left) / L      ← variación de orientación
```

Donde L es la distancia entre ruedas (track width) del robot.

8.3 Actualización de posición

```
x = x + d_center × cos(θ)  
y = y + d_center × sin(θ)  
θ = θ + delta_θ
```

9. Instalación y Ejecución

Requisitos previos: Node.js LTS.

1. Instalar dependencias:

```
npm install
```

2. Iniciar el servidor de desarrollo:

```
npm run dev
```

3. Abrir `http://localhost:3000` en el navegador.

Nota: El dashboard de odometría funciona sin configurar ninguna variable de entorno. La clave `GEMINI_API_KEY` es opcional y solo es necesaria si se desea utilizar el asistente de inteligencia artificial integrado con Google Gemini.

Comando	Descripción
<code>npm run dev</code>	Servidor Express + Vite en modo desarrollo (HMR activo)
<code>npm run build</code>	Genera bundle de producción en <code>dist/</code>
<code>npm run preview</code>	Previsualiza el build de producción
<code>npm run lint</code>	Verificación de tipos con TypeScript
<code>npm run clean</code>	Elimina la carpeta <code>dist/</code>

10. Conclusiones

El proyecto Wheels Odometry demuestra la aplicación práctica de conceptos de robótica móvil en un entorno de desarrollo web moderno con TypeScript. La integración de Express como backend y React como frontend produce un sistema de monitoreo responsivo, extensible y de bajo costo computacional.

El uso de TypeScript en todo el stack garantiza la seguridad de tipos y facilita el mantenimiento del código. La arquitectura SPA con API REST permite que el frontend sea independiente del origen de los datos, por lo que conectar el sistema a encoders físicos reales (vía puerto serial, WebSocket o MQTT) no requeriría modificar la interfaz de usuario.

Como trabajo futuro se propone: integración con hardware real mediante ESP32 o Arduino, implementación de WebSockets para reducir latencia, exportación de trayectorias en CSV o JSON, corrección de deriva odométrica con fusión sensorial (IMU), y mejora del asistente de IA con Gemini para análisis de trayectorias.

11. Referencias

- Siegwart, R., Nourbakhsh, I. R., & Scaramuzza, D. (2011). Introduction to Autonomous Mobile Robots (2.a ed.). MIT Press.
- Expressjs.com. (2024). Express – Fast, unopinionated web framework for Node.js. <https://expressjs.com>

- React. (2024). React – The library for web and native user interfaces. <https://react.dev>
- Vite. (2024). Vite – Next Generation Frontend Tooling. <https://vitejs.dev>
- Chart.js. (2024). Chart.js – Simple yet flexible JavaScript charting. <https://www.chartjs.org>
- TypeScript. (2024). TypeScript – JavaScript with syntax for types.
<https://www.typescriptlang.org>
- Tailwind CSS. (2024). Tailwind CSS – A utility-first CSS framework. <https://tailwindcss.com>